Chapter Three:Processes

Compiled By:Bekretsyon B.(Msc)

April 7, 2022

Outline

- Threads
  - Introduction to Threads
  - Thread Implementation
  - Threads in Distributed Systems: Multithreaded Clients
- Client
  - User Interfaces
- Servers
  - General Organization
  - Types of Servers
  - Out-of-Band Communication
  - Servers and State
  - Server Clusters
- Code migration
  - Approaches to Code Migration
  - Migration and Local Resources
  - Migration in Heterogeneous Systems

Introduction to Threads

Inroduction

- Process is an instance of a computer program that is being executed. It contains the program code and its current activity.
- Communication takes place between processes
- A process is a program in execution.
- From OS perspective, management and scheduling of processes is important
- Other important issues arise in distributed systems:
  - Multithreading to enhance performance
  - How are clients and servers organized
  - Process or code migration to achieve scalability and to dynamically configure clients and servers

Introduction to Threads

We build virtual processors in software, on top of physical processors:

Processor:

- Provides a set of instructions along with the capability of automatically executing a series of those instructions.

Thread:

- A minimal software processor in whose context a series of instructions can be executed.
- Can be seen as execution of (part of) a program on a virtual processor.
- Analogous to the concept of a procedure that runs independently from its main program.
- Useful to structure large applications into parts that could logically be executed at the same time.

Introduction to Threads

Process:

- A software processor in whose context one or more threads may be executed.
- A program that is currently being executed on one of the OSs virtual processors.

- Management and scheduling of processes are the most important issues in operating systems.
- In addition, multithreading and process (code) migration are also important issues in distributed systems.
- Multithreading useful to efficiently organize client server systems.
- Code migration:
  - Helps to achieve scalability.
  - Helps to dynamically configure clients and servers.

Introduction to Threads

- Threads can be used in both distributed and non DS systems
- Threads allow multiple executions to take place in the same process environment, called Multithreading

Thread Usage – Why do we need threads?

- E.g., a wordprocessor has different parts;
  - Parts for interacting with the user, formatting the page as soon as changes are made timed savings (for auto recovery), spelling and grammar checking, etc.
- Simplifying the programming model: since many activities are going on at once
- They are easier to create and destroy than processes since they do not have any resources attached to them
- Performance improves by overlapping activities if there is too much I/O; i.e., to avoid blocking when waiting for input or doing calculations, say in a spreadsheet
- Real parallelism is possible in a multiprocessor system

Introduction to Threads: Multithreading

Advantages of multithreading

- No need to block with every system call.
  - In a single-threaded process, whenever a blocking system call is executed, the process as a whole is blocked. E.g. a spreadsheet cell will be unable to process cell-dependent computations while the program is waiting for input.
- Easy to exploit available parallelism in multiprocessors
  - It becomes possible to exploit parallelism when executing a program on a multiprocessor system.
    - Each thread is assigned to a different CPU.
    - Shared data are stored in shared main memory

Introduction to Threads: Multithreading

Advantages of multithreading

- Cheaper communication between components than with IPC
  - In stead of using processes, large applications can be constructed such that different parts are executed by separate threads
    - Communication between those parts is dealt with by using shared data.
    - Thread switching can sometimes be done entirely in user space.
- Software engineering expressive power
  - From software engineering perspective, many applications are simply easier to structure as a collection of cooperating threads.
    - For example, in the case of word processor, separate threads can be used for handling user input, spelling and grammar checking, etc.

Thread Implementation

Thread Package

- Threads are generally provided in the form of a thread package.
- Thread package contains:
  - Operations to create and destroy threads.
  - Operations on synchronization variables (mutexes and conditional variables).
- Three approaches to thread package implementation:
  - User-Level Solution
  - Kernel-Level Solution
  - Lightweight Process (LWP) Solution

Threads in Distributed Systems

- Multithreaded Clients
- Multithreaded Servers: Servers can be constructed in three ways
  - Single-threaded process-it gets a request, examines it, carries it out to completion before getting the next request
  - Threads
  - Finite-state machine

Threads in Distributed Systems: Multithreaded Clients

Multithreaded Clients

- Main issue is hiding network latency.
  - Initiate communication and immediately proceed with something else.
- Examples:
  - Multithreaded web clients
    - Browser scans HTML, and finds more files that need to be fetched.
    - Each file is fetched by a separate thread, each issuing an HTTP request.
    - As files come in, the browser displays them.
    - Several connection can be opened simultaneously.
  - Multiple request-response calls to other machines (RPC)
    - A client does several calls at the same time, each one by a different thread.
    - It then waits until all results have been returned.
    - If calls are to different servers, we may have a linear speed-up.

Threads in Distributed Systems: Multithreaded Servers

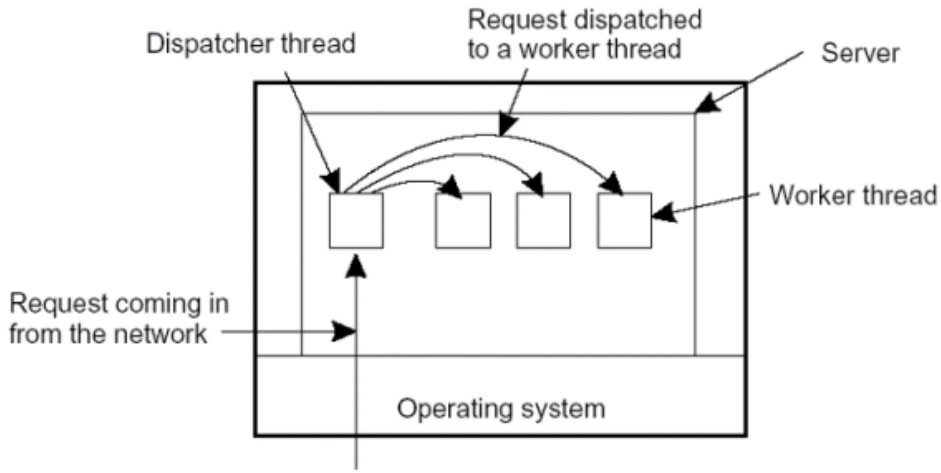
Multithreaded Servers

- An important use of multithreading in distributed systems is at the server side.
- Main issues are:
  - Improving performance
    - Starting a thread to handle an incoming request is much cheaper than starting a process.
    - Exploits parallelism to attain high performance. Single-threaded server cant take advantage of multiprocessor.
    - Hides network latency. Other work can be done while a request is coming in.
  - Better structure
    - Most servers have high I/O demands. Using simple, well-understood blocking calls simplifies the overall structure.
    - Multithreaded programs tend to be smaller and easier to understand due to simplified flow of control.

Threads in Distributed Systems: Multithreaded Servers

Multithreaded Server Organization

- Dispatcher thread reads incoming requests.
- Worker thread is selected by the server to process a request



A Multithreaded server organized in a dispatcher/worker model

Threads in Distributed Systems: Finite State Machine

- A state machine consists of
  - State variables, which encode its state
  - commands, which transform its state
- Each command is implemented by a deterministic program
  - If threads are not available
  - It gets a request, examines it, tries to fulfill the request from cache, else sends a request to the file system;
  - But instead of blocking it records the state of the current request and proceeds to the next request output
- Operation:
  - Using a single process that awaits messages containing requests and performs the actions they specify, as in a server.
    - Read any available input.
    - Process the input chunk.
    - Save state of that request.
    - Loop

Threads in Distributed Systems: Finite State Machine

Three ways to construct a server...

Server Model	Parallelism	Performance	System Calls	Processing	Programming
Multithreaded	Parallel	High	Blocking	Sequential	Easy
Single-threaded	Non-parallel	Low	Blocking	Sequential	Easy
Finite-state machine	Parallel	High	Non-blocking	Non-sequential	Hard

User Interfaces

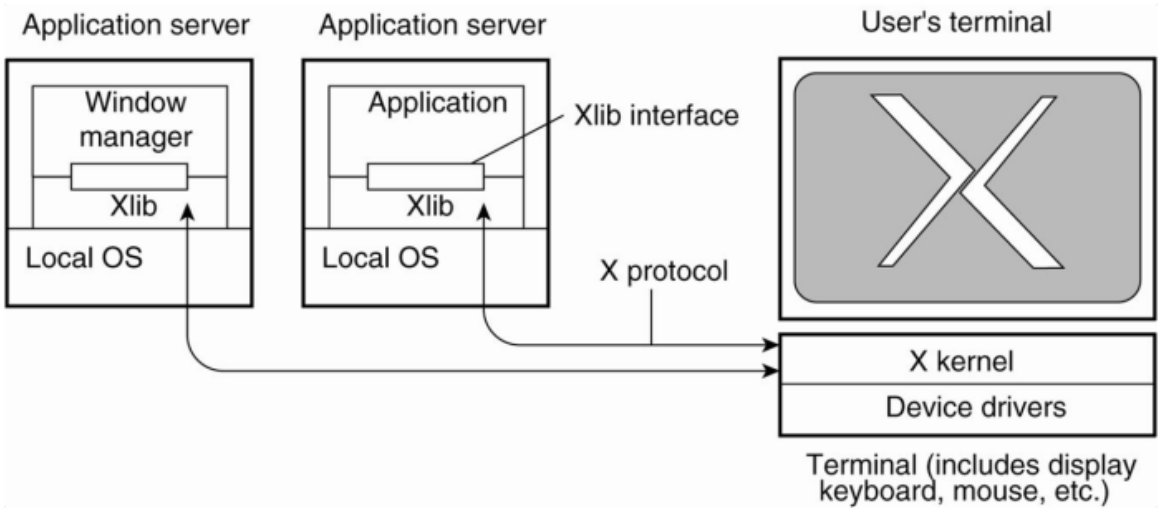
Two issues: user interfaces and client-side software for distribution transparency

User Interfaces

- To create a convenient environment for the interaction of a human user and a remote server; e.g. mobile phones with simple displays and a set of keys
- GUIs are most commonly used

User Interfaces: The X window system

- The X Window System (or simply X)
  - It has theX-kernel: the part of the OS that controls the terminal (monitor, keyboard, pointing device like a mouse)
  - Contains all terminal-specific device drivers through the library called xlib

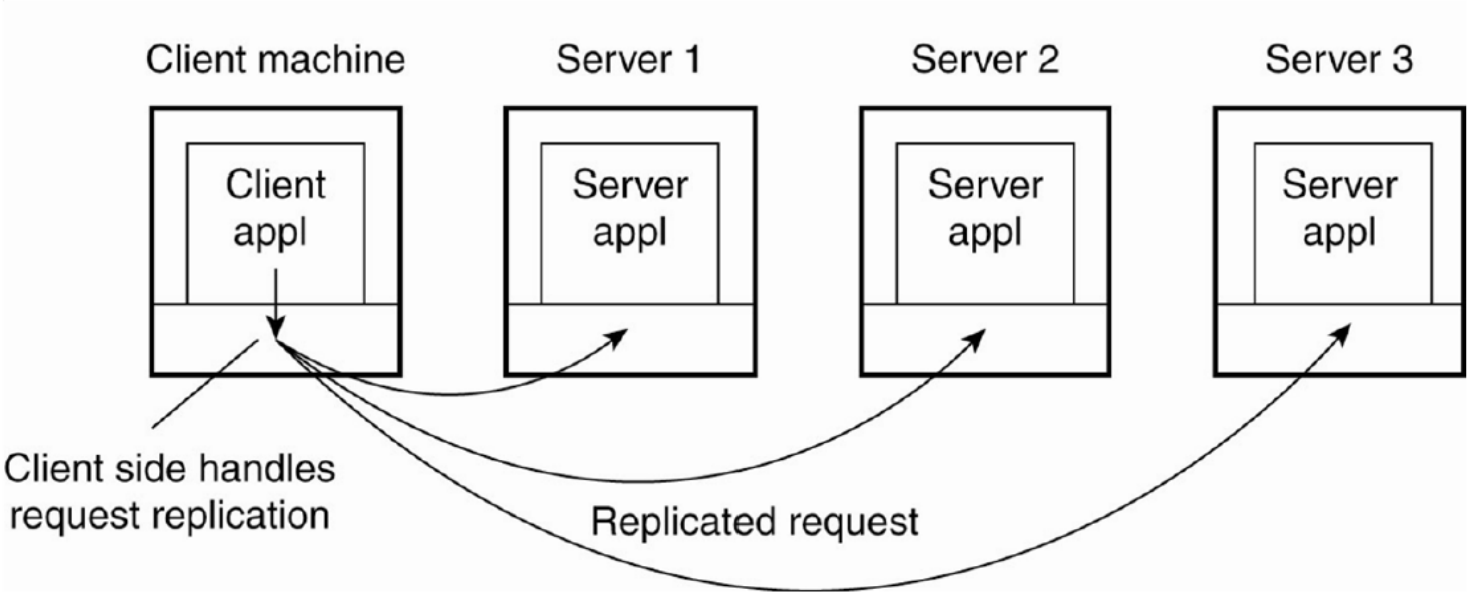


The basic organization of the X Window System

Client-Side Software for Distribution Transparency

- In addition to the user interface, parts of the processing and data level in a client-server application are executed at the client side
- An example is embedded client software for ATMs, cash registers, etc.
- Moreover, client software can also include components to achieve distribution transparency
- Access transparency: client-side stubs for machine architectures and communications.
- Location/migration transparency: Let client-side software keep track of actual location
- Failure transparency: Can often be placed only at client. E.g., using caches in browsing.
- Replication transparency: Multiple invocations handled by client-side stub

Client-Side Software for Distribution Transparency



Transparent replication of a server using a client-side solution

- Access transparency and failure transparency can also be achieved using client-side software

Servers and design issues

General Design Issues

- How to organize servers?
- Where do clients contact a server?
- Whether and how a server can be interrupted
- Whether or not the server is stateless

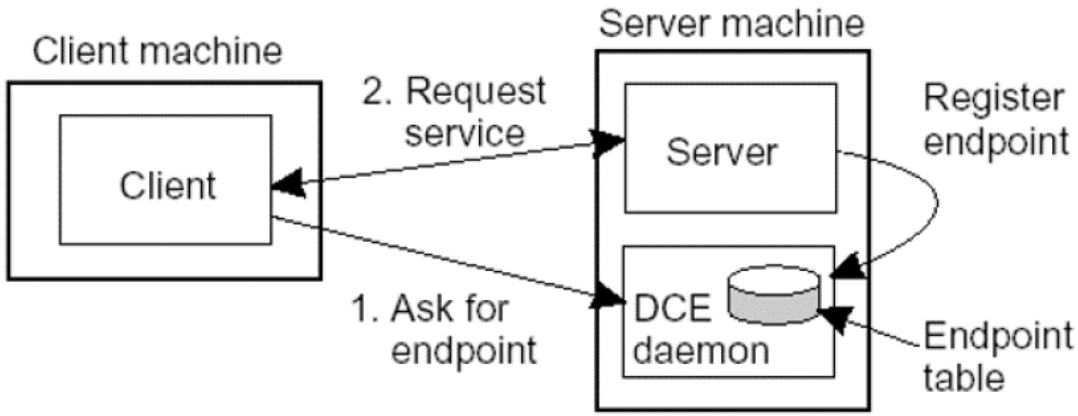
General Organization

- A server is a process that waits for incoming service requests at a specific transport address.
- Clients send requests to an endpoint, also called a port.
- In practice, there is a one-to-one mapping between a port and a service
  - 20 ftp-data File Transfer [Default Data]
  - 21 ftp File Transfer [Control]
  - 23 telnet Telnet
  - 25 smtp Simple Mail Transfer Protocol
  - 53 dns Domain Name System
  - 80 http Hypertext Transfer Protocol
- For services that do not require pre assigned endpoints, it can be dynamically assigned by the local OS

General Organization

How can the client know this endpoint? Two approaches

- Have a daemon running and listening to a well-known endpoint like in DCE; it keeps track of all endpoints of services on the collocated server
  - The client will first contact the daemon which provides it with the endpoint, and then the client contacts the specific server

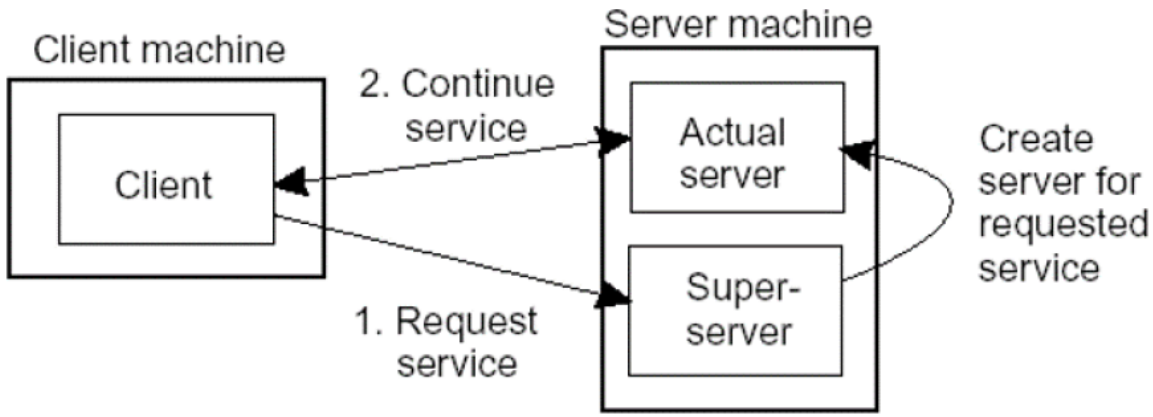


Client-to-server binding using a daemon as in DCE

General Organization

How can the client know this endpoint? Two approaches

- Use a superserver (as in UNIX) that listens to all endpoints and then forks a process to take care of the request; this is instead of having a lot of servers running simultaneously and most of them idle Client-



Client-to-Server binding using a superserver as in UNIX

Types of Servers

Types of Servers

- Iterative vs. Concurrent Server:
  - Iterative server itself handles only one request at a time whereas
  - concurrent server passes the request to a separate thread or another process and waits for another request.
- Superservers:
  - Servers that listen to several ports, i.e., provide several independent services.
  - When a service request comes in, they start a subprocess to handle the request.
  - For services with more permanent traffic, get a dedicated server.

Out-of-Band Communication

How can a server be interrupted once it has accepted (or is in the process of accepting) a service request?

- For instance, a user may want to interrupt a file transfer, may be it was the wrong file
- Let the client exit the client application; this will break the connection to the server; the server will tear down the connection assuming that the client had crashed
- Solution 1: Use a separate port for urgent data (possibly per service request):
  - Server has a separate thread (or process) waiting for incoming urgent messages.
  - When urgent message comes in, associated request is put on hold.
  - Note: we require OS supports high-priority scheduling of specific threads or processes.
- Solution 2: Use out-of-band communication facilities of the transport layer:
  - Ex, TCP allows to send urgent messages in the same connection.
  - Urgent messages can be caught using OS signaling techniques.

Servers and State

Stateless Servers

- Never keep accurate information about the status of a client after having handled a request:
  - Dont record whether a file has been opened (simply close it again after access)
  - Dont promise to invalidate a clients cache
  - Dont keep track of your clients
- Consequences:
  - Clients and servers are completely independent
  - State inconsistencies due to client or server crashes are reduced
  - Possible loss of performance because, e.g., a server cannot anticipate client behavior (think of prefetching file blocks)

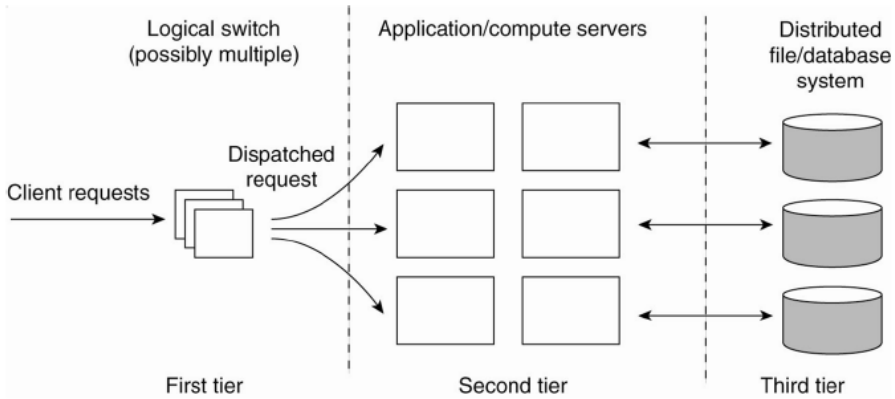
Servers and State

Stateful Servers

- Keeps track of the status of its clients:
  - Record that a file has been opened, so that prefetching can be done
  - Knows which data a client has cached, and allows clients to keep local copies of shared data
  - Server may record clients behavior using cookies
- Observation:
  - The performance of stateful servers can be extremely high, provided clients are allowed to keep local copies.
  - Session state vs. permanent state
  - As it turns out, reliability is not a major problem.

Server Clusters

- Server cluster is collection of machines connected through a network, where each machine runs one or more servers.
- Many server clusters are organized along three different tiers, to improve performance.
- The first tier is generally responsible for passing requests to an appropriate server.
- The second and the third tiers can be merged.

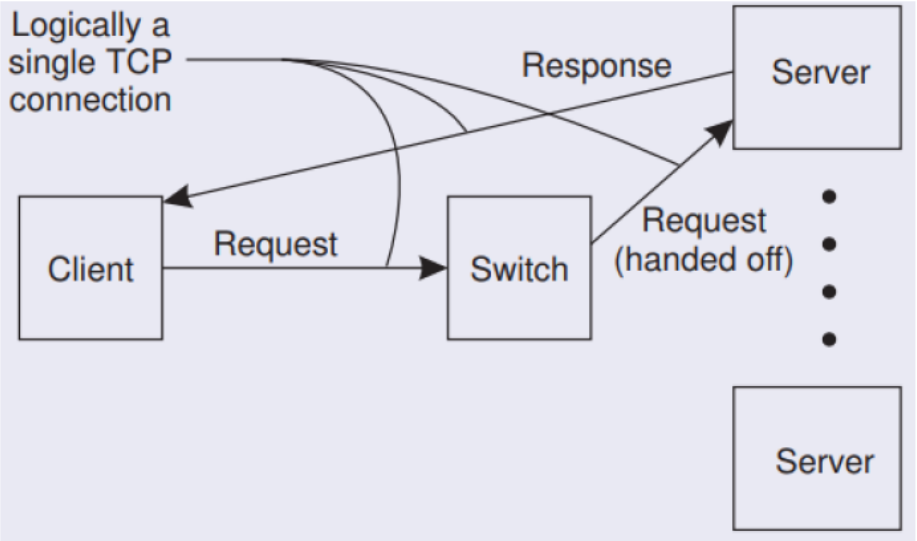


The general organization of a three-tiered server cluster

Server Clusters

Request Handling

- Having the first tier handle all communication from/to the cluster may lead to a bottleneck.
- Solution: Various, but one popular one is TCP-handoff:



Code migration

- So far, communication was concerned on passing data
- We may pass programs, even while running and in heterogeneous systems
- Code migration also involves moving data as well: when a program migrates while running, its status, pending signals, and other environment variables such as the stack and the program counter also have to be moved

Reasons for Migrating Code

- To improve performance; move processes from heavily-loaded to lightly-loaded machines (load balancing)
- To reduce communication: move a client application that performs many database operations to a server if the database resides on the server; then send only results to the client
- To exploit parallelism (for nonparallel programs): e.g., copies of a mobile program moving from site to site searching the Web

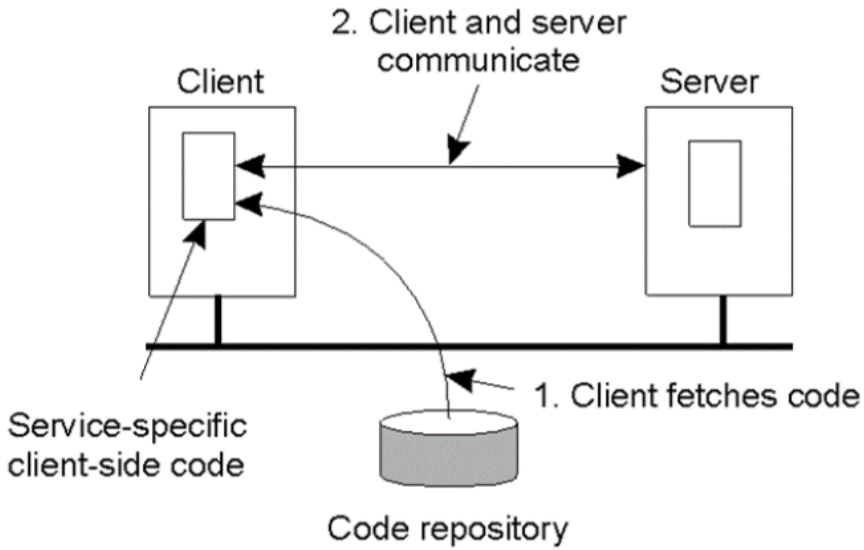
Approaches to Code Migration

- A process consists of three segments
  - Code segment: contains the actual code
  - Resource segment: contains references to external resources needed by the process. E.g., files, printers, devices, other processes
  - Execution segment: stores the current execution state of a process, consisting of private data, the stack, and the program counter
- What if operation we want (e.g., moving data) isnt offered remotely?
  - Solution: Code Migration / Agents
  - Traditionally, code is moved along with the whole computational context
  - Why code migration is useful?
    - Load balancing
    - Minimizing communication costs
    - Optimizing perceived performance
    - Improving scalability
    - Flexibility through dynamic configurability

Approaches to Code Migration

Dynamic Client Configuration

- The client first fetches the necessary software, and then invokes the server.



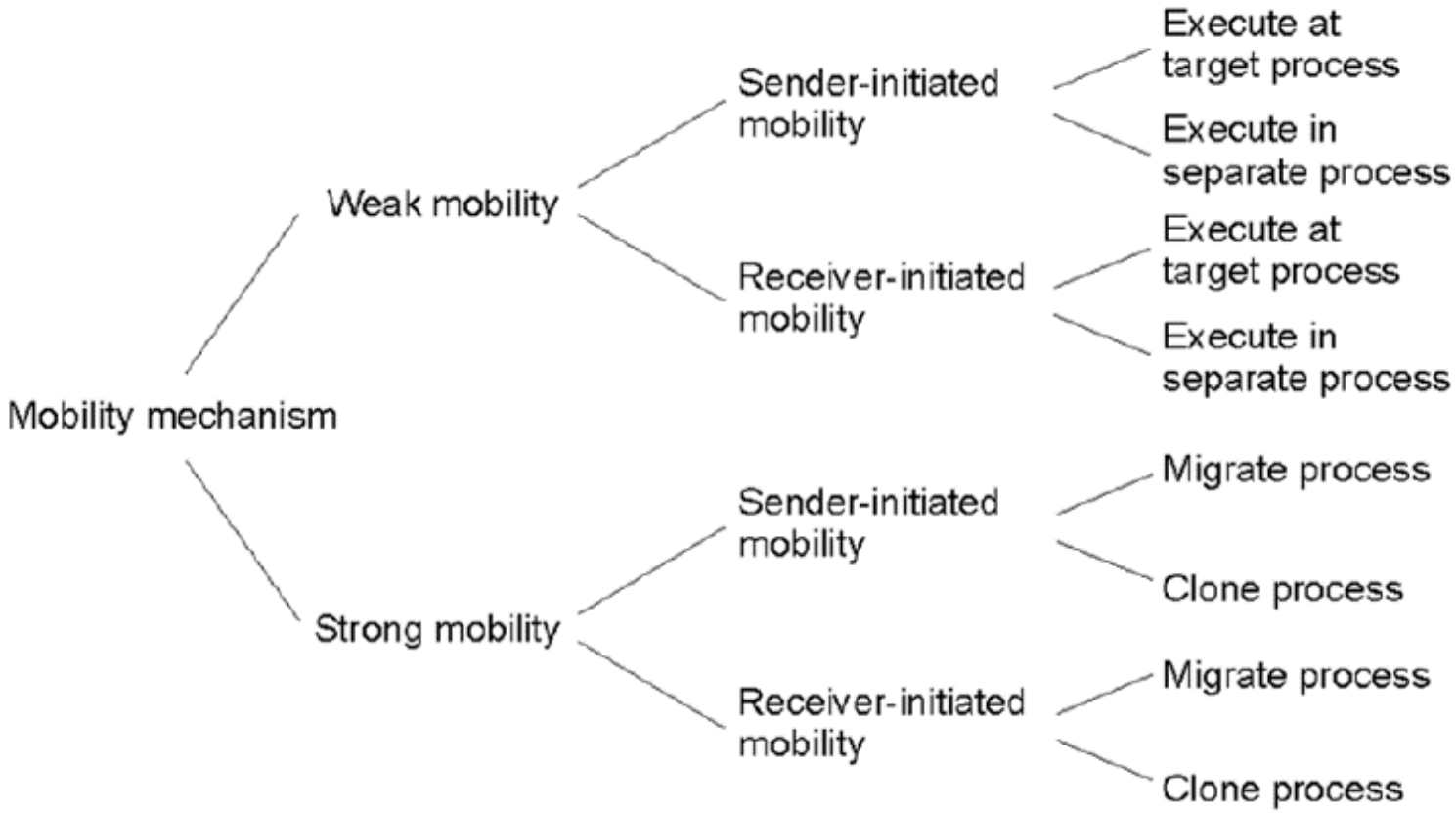
The principle of dynamically configuring a client to communicate to a server; the client first fetches the necessary software, and then invokes the server

Approaches to Code Migration

- Mobility
  - Weak Mobility: Moves only code, plus maybe some init data (and start execution from the beginning after migration).
    - The mobile code may be executed on either the target process or a separate process.
    - E.g., Java Applets.
  - Strong Mobility: Moves code and execution (and resume execution where it stopped).
    - Migration: move the entire object from one machine to the other.
    - Cloning: simply start a clone, and set it in the same execution state.
- Initiation
  - Sender-initiated migration: Migration is initiated where the code resides / is being currently executed.
  - Receiver-initiated migration: Migration is initiated by the target machine.

Approaches to Code Migration

Alternatives of Code Migrations



Migration and Local Resources

- So far, we have only accounted for migration of the code and execution segments.
- A process uses local resources that may or may not be available at the target site.
- Either references need to be updated, or resources need to be moved.
- Resources might not be as easy to move around as code and variables.
  - Example: A huge database might in theory be moved across the network, but in practice it will not.
- Two issues:
  - How does the resource segment refer to resources?
    - Process-to-resource binding
  - How does the resource relate with the hosting machine?
    - Resource-to-machine binding

Migration and Local Resources

How does the resource segment refer to resources?

Process-to-Resource Binding

- Binding by identifier: the process refers to a resource by its identifier (e.g., a URL).
  - Bind to the same resource
- Binding by value: the object requires the value of a resource (e.g., standard library).
  - Bind to equivalent resource
- Binding by type: the object requires a type of resource (e.g., local devices, such as monitors, printers, and so on).
  - Bind to resource with same function

Migration and Local Resources

- When migrating code, we may need to change the references to resources but cannot change the kind of process-to-resource binding.
- How a resource reference is changed depends on the resource-to-machine bindings

How does the resource relate with the hosting machine?

Resource-to-Machine Binding

- Unattached: Resources that can be easily moved between different machines.
  - E.g., data files and cache.
- Fastened: Resources that can be moved, but only at a high cost.
  - E.g., local databases and complete Web sites.
- Fixed: Resources bounded to a specific machine.
  - E.g., a local communication port and monitor.

Migration and Local Resources

Actions to be Taken...

- Actions to be taken with respect to the references to local resources when migrating code to another machine.

Process-to-resource binding	Resource-to machine binding: Unattached	Fastened	Fixed
By identifier	MV (or GR)	GR (or MV)	GR
By value	CP (or MV, GR)	GR (or CP)	GR
By type	RB (or GR, CP)	RB (or GR, CP)	RB (or GR)

Actions to be taken with respect to the references to local resources when migrating code to another machine

- GR: Establish a global systemwide reference
- CP: Copy the value of the resource
- RB: Rebind the process to a locally available resource
- MV: Move the resource

Migration in Heterogeneous Systems

- Main problem:
  - The target machine may not be suitable to execute the migrated code.
  - The definition of process/thread/processor context is highly dependent on local hardware, operating system and runtime system.
- Solution
  - Make use of an abstract machine that is implemented on different platforms.
    - Interpreted languages running on a virtual machine E.g., Java/JVM, scripting languages
    - Virtual machine monitors allowing migration of complete OS + apps.

Any Question?